How components get their data

This is a map of where screen data comes from in Zafira React. Use it when you open a page and need to know which file loads the records, which API it calls, and which pieces never call the server at all.

The path every live screen uses

```
Page (useEffect)
  → function in frontend/services/*.js
    → fetch('/api/...')
      → Vite proxy (frontend/vite.config.js) forwards /api to http://localhost:3000
        → backend/src/routes/api.js mounts the module router
          → route handler
            → backend/src/services/*Db.js or *Service.js
              → MySQL when backend/.env is set
              → in-memory mock rows when the database is not configured
```

There is no Redux, no React Query, and no global data store. A page keeps what it loaded in `useState`. Child sections, tables, and modals receive that state as props. A few small widgets fetch on their own (vessel search, port search, distance lookup). Those are called out below.

Login stores a bearer token in `localStorage`. `frontend/src/main.jsx` installs `setupAuthFetch()`, which wraps `window.fetch` and adds `Authorization: Bearer <token>` to every `/api` call. Service files do not attach the token themselves.

Three ways a component gets data

Kind	What it means	Where you look
API	The page (or a search widget) calls `frontend/services`	The service file's `BASE` constant, then the matching backend route
Props	A parent already loaded the record and passed fields down	The parent page's `useEffect` and the props on the child
Local	Constants, URL query, `sessionStorage`, or math on data already in the browser	The file next to the component (`*Mock.js`, `*Model.js`, `estimateCalculations.js`)

URL query values are a real data source for voyage screens. Ops pages read `comId` (and often `page`, `voyageNo`) from the address bar, then pass that id into the fetch. Glance hubs read `tab` and only choose which list page to render.

Shell and session (every internal page)

These load outside the page you are looking at.

UI	Source	API
Login	`frontend/pages/LoginPage.jsx` calls `login()` from `@bainbridge/shared-auth`	`POST /api/auth/login`
Agent login	`frontend/pages/agent/AgentLoginPage.jsx`	`POST /api/auth/agent-login`
Signed-in user name	`getUser()` reads `localStorage` (`auth_user_internal`)	Written at login. No extra fetch
Header recent work	`AppHeader` calls `fetchRecentWork()`	`GET /api/recent_work`
Header alerts	`AppHeader` calls `fetchUserAlerts()`	`GET /api/alerts`
Alert refresh after a save	Pages call `notifyAlertsUpdated()` / `notifyRecentWorkUpdated()`, which fire window events the header listens for	Same two endpoints
Sidebar open/closed	`localStorage`	None
Selected business type	`frontend/utils/businessTypeSession.js` (`sessionStorage`), then sent as `selBType` on list calls	The list API, not a separate store
Ops list filters	`frontend/pages/internal-user/ops/opsVcListFilters.js`	Filters are local. The list fetch uses them as query params

UI	Source	API
Period-contract list status	`frontend/utils/periodContractListSession.js`	Same pattern
Home module cards	Hard-coded `HOME_CARDS` in `ModuleHomePage.jsx`. The only live bit is the logged-in user	None for the cards

Shared UI (`Button`, `CardSelect`, `FilterBar`, tables) does not load business data. `PageHeaderContext` only carries the heading text the layout should show.

Screens that do not call the API

Screen	Data
Module home	Static card list
User guide list and viewer	Titles in `frontend/constants/userGuides.js`. The PDF is a static file under `/attachment/<fileName>`
VC dashboard charts (`DashboardDemoCharts`, spark cards, desk mix)	Sample numbers in `frontend/pages/internal-user/vc/dashboardDemoData.js`. The badge on those charts says sample data. The voyage tables on the same page are live (see VC dashboard below)
Daily Position report	`frontend/pages/internal-user/reports/dailyPositionsMock.js`
Aging receivables / payables	`AR_ROWS_RAW` and `AP_ROWS_RAW` inside `AgingReportPage.jsx`
Master placeholder, SOPF coming soon	Static copy
`OpsVcGlanceHubPage`	No fetch. It reads `?tab=` and renders In Ops, Post Ops, or History
Confirm dialogs, page headers, layout chrome	Local React state

Estimate and TC **totals** (TCE, hire days, freight) are calculated in the browser from the form state (`estimateCalculations.js`, `calcTcTotals` in `tcEstimates.js`, `opsVcPaymentGridModel.js`). The inputs still come from the API. The math does not.

How to trace one screen

1. Open the page file under `frontend/pages/`.
2. Find the `from '../../../services/...'` import. That file is the data source.
3. Read `const BASE` at the top of the service. That is the API prefix.
4. Open `backend/src/routes/api.js` and follow that prefix to the route file.
5. The route calls a `*Db.js` or `*Service.js` under `backend/src/services/`.

Example: Payment Grid reads `comId` from the URL, calls `fetchPaymentGridVc(comId)` in `frontend/services/opsVc.js`, which hits `/api/internal-user/vc/...`. The grid sections and money chips are derived in `opsVcPaymentGridModel.js` from that response. They are not a second request.

Catalog

Paths below are under `frontend/` unless noted. “Reads” is the function the page calls to load. Saves use the matching `create*` / `update*` / `save*` in the same service file.

SOPF (estimates)

Service files: `services/estimateList.js`, `services/estimateDetail.js`, `services/supportTickets.js`, `services/vesselPositions.js`.
API prefix: `/api/internal-user/sopf`.
Backend route: `backend/src/routes/sopf.js`.

Page	Reads
`pages/internal-user/sopf/EstimateListPage.jsx`	`fetchEstimateList`, `fetchBusinessTypes`

Page	Reads
`AddEstimatePage.jsx`	`fetchEstimateLookups`, `fetchPeriodPrefill`, `fetchNextEstimateNo`. Also `fetchCoa` and `fetchPeriodContract` when the new estimate is opened from a COA or period
`UpdateEstimatePage.jsx` / `ViewEstimatePage.jsx`	`fetchEstimateDetail`, `fetchEstimateLookups`, `fetchEstimateVcIn`
`EstimateDetailSections.jsx`	Parent passes the estimate. This file calls `searchEstimatePorts`, `fetchCanalOrcRates`, `checkVoyageNoExists` while the user edits
`VesselSearchSelect.jsx`	`searchVessels` as the user types
`DistanceFetchModal.jsx`	`fetchPortDistance`
`TankerFreightModeSection.jsx`	`searchEstimatePorts`
`VesselPositionPage.jsx`	`fetchVesselsWithinRange`
`SupportTicketPage.jsx`	`fetchSupportTickets`, `fetchTicketMessages`

`pages/internal-user/ops/voyage-financials/CostSheetEstimatePage.jsx` and `CostSheetDetailSections.jsx` reuse the same SOPF estimate services. They are ops screens sitting on estimate data.

VC dashboard

Service: `services/vcDashboard.js`. API: `/api/internal-user/vc`.

Page	Reads
`pages/internal-user/vc/VcDashboardPage.jsx`	`fetchVcDashboardMeta`, `fetchVcBusinessDashboard`, `fetchTcBusinessDashboard`, `fetchCoaList`, `fetchCoaBusinessOverview`, `fetchPeriodList`, `fetchCoaShipments`, `fetchPerformingVessels`
`DashboardDemoCharts.jsx`	Demo file only. See the table above

`fetchVcBusinessTypes` from this same service is the business-type dropdown on many other lists (COA, fleet, period, ops glance, cargo relet, todo voyage search).

VC operations

Service: `services/opsVc.js`. API: `/api/internal-user/vc`.
Almost every page takes `comId` from the query string.

Page	Reads
`OpsVcInOpsGlancePage.jsx`	`fetchInOpsAtGlance`
`OpsVcPostOpsPage.jsx`	`fetchPostOpsAtGlance`
`OpsVcHistoryPage.jsx`	`fetchHistoryAtGlance`
`OpsVcYearUpdationPage.jsx`	`fetchYearUpdation`
`OpsVcVoyageReportPage.jsx`	`fetchVoyageReports`
`OpsVoyageStatusModal.jsx`	`fetchVoyageStatus`
`OpsVcPaymentGridPage.jsx`	`fetchPaymentGridVc`
`OpsVcFreightInvoicePage.jsx`	`fetchFreightInvoiceForm`, `fetchFreightInvoiceBanking`
`OpsVcRequestPortCostPage.jsx`	`fetchRequestPortCostForm`, `fetchRequestPortCostVendorBanking`
`OpsVcOtherInvoicePage.jsx`	`fetchOtherInvoiceForm`, `fetchOtherInvoiceBanking`
`OpsVcHireStatementPage.jsx`	`fetchHireStatementForm`
`OpsVcClubbedInvoicePage.jsx`	`fetchClubbedFreightInvoice`
`OpsVcClubbedHirePage.jsx`	`fetchClubbedHireInvoice`
`OpsVcSofPage.jsx`	`fetchSofForm`
`OpsVcLaytimePage.jsx`	`fetchLaytimeForm`

Page	Reads
`OpsVcBunkerPage.jsx`	`fetchBunkerForm`
`OpsVcSoaReportPage.jsx`	`fetchSoaReport`
`OpsVcDocumentsPage.jsx`	`fetchOpsDocuments`
`OpsVcAgencyLetterPage.jsx`	`fetchAgencyLetterForm`
`OpsVcPdaFdaPage.jsx`	`fetchOpsPdaFda`
`OpsVcChecklistPage.jsx`	`fetchOpsChecklist`
`OpsVcCostSheetPage.jsx`	`fetchOpsVcCostSheet`
`OpsVcCostSheetInPage.jsx`	`fetchOpsVcCostSheetIn`
`OpsVcCompareSheetsModal.jsx`	`fetchCompareSheetsVc`

Header action components (`OpsVcPaymentGridHeaderActions`, `VoyageOpsHeading`, and the rest) do not fetch. They receive labels and links from the page.

TC estimates and TC operations

TC estimates — service `services/tcEstimates.js`, API `/api/internal-user/tc-estimates`, route `backend/src/routes/tcEstimates.js`.

Page	Reads
`tc/TcOutEstimatesListPage.jsx`	`fetchTcEstimates`, `fetchTcBusinessTypes`
`tc/TcFixtureFormPage.jsx`	`fetchTcLookups`, `fetchTcEstimate`, `fetchNextTcEstimateNo`. Vessel prefill comes from `fetchVesselEstimatePrefill` (SOPF). A linked period comes from `fetchPeriodContract`
`tc/TcViewPage.jsx`	`fetchTcEstimate`
`tc/TcCalculatePage.jsx`	`fetchTcEstimate`. Totals then run through `calcTcTotals` in the browser
`tc/TcDecisionChartsListPage.jsx`	`fetchTcDecisionCharts`
`tc/TcDecisionChartModal.jsx`	`fetchTcCompareEstimates`
`tc/TcDecisionChartDetailsModal.jsx`	`fetchTcDecisionChartDetails`
`tc/TcInExpensesModal.jsx`	No fetch. Date helpers from `tcEstimates.js` only

TC ops — service `services/opsTc.js`, API `/api/internal-user/vc` (same VC router as voyage ops).

Page	Reads
`ops/OpsTcInOpsGlancePage.jsx`	`fetchInOpsAtGlanceTc`
`ops/OpsTcPostOpsPage.jsx`	`fetchPostOpsAtGlanceTc`
`ops/OpsTcHistoryPage.jsx`	`fetchHistoryAtGlanceTc`
`ops/OpsTcYearUpdationPage.jsx`	`fetchYearUpdationTc`
`ops/OpsTcFinalisedFixturesPage.jsx`	`fetchFinalisedVoyageFixturesTc`
`ops/OpsTcFixtureNotePage.jsx`	`fetchOpsTcFixtureNote`
`ops/OpsTcChecklistPage.jsx`	`fetchTcChecklist`
`ops/OpsTcCostSheetPage.jsx`	`fetchOpsTcCostSheet`, plus `fetchTcEstimate` for the fixture
`ops/OpsTcAgencyLetterPage.jsx`	`fetchAgencyLetterTcForm`
`ops/OpsTcDocumentsPage.jsx`	`fetchOpsTcDocuments`
`ops/OpsTcPaymentGridPage.jsx`	`fetchPaymentGridTc`
`ops/OpsTcCompareSheetsModal.jsx`	`fetchCompareSheetsTc`

`OpsTcPage.jsx` is a router shell around those pages. It does not load records itself.

COA and cargo relet

COA — `services/coas.js`, API `/api/internal-user/coa`.
Standalone cargo relet — `services/cargoRelets.js`, API `/api/internal-user/cargo-relets`.

Page	Reads
`coa/RunningCoasListPage.jsx`	`fetchRunningCoas`, `fetchVcBusinessTypes`
`coa/CoaFormPage.jsx`	`fetchCoaLookups`, `fetchCoa`, `fetchCoaNominations`
`coa/CoaOpsListPage.jsx` (`CoaInOpsPage`, `CoaPostOpsPage` use this)	COA list functions plus ops status helpers from `opsVc.js`
`coa/CargoReletListPage.jsx`	`fetchCargoRelets` from `coas.js` (COA-scoped relets)
`coa/CargoReletFormPage.jsx`	COA relet functions, `fetchStandaloneCargoRelet` when needed, `fetchCommercialParameters`, `fetchVesselEstimatePrefill`
`cargo-relet/CargoReletListPage.jsx`	`fetchStandaloneCargoRelets`

Form section components under `coa/` receive the loaded COA as props.

Period contracts

Service: `services/periodContracts.js`. API: `/api/internal-user/period-contracts`.

Page	Reads
`period-contract/PeriodContractListPage.jsx`	`fetchPeriodContractList`, `fetchVcBusinessTypes`
`period-contract/AddPeriodContractPage.jsx`	`fetchPeriodContractLookups`, `fetchPeriodContract`, `fetchPeriodLinkedVoyage`, `fetchPeriodNominations`, `fetchVesselEstimatePrefill`
`period-contract/PortSearchSelect.jsx`	`searchPeriodContractPorts`
`AddPeriodContractSections.jsx`	Props from the page

Fleet

Services: `services/fleet.js`, `services/vesselTankerParticulars.js`, `services/commercialParameters.js`.
API: `/api/internal-user/fleet`.

Page	Reads
`fleet/FleetPage.jsx`	`fetchFleetList`, `fetchFleetCompare`, `fetchVcBusinessTypes`
`fleet/AddVesselPage.jsx`	`fetchVesselPrimaryLookups`
`fleet/UpdateVesselPage.jsx`	`fetchVesselPrimary`
`fleet/ViewVesselTankersPage.jsx`	`fetchTankerParticulars`
`fleet/UpdateVesselTankersPage.jsx`	`fetchTankerParticulars`
`fleet/CommercialParametersPage.jsx`	`fetchCommercialParameters`

Reports

Service: `services/reports.js`. API: `/api/internal-user/reports`.
`ReportModulePage.jsx` does not fetch. It reads `:reportId` from the route, looks up `frontend/constants/reportsDefinitions.js`, and renders one of the pages below. PDF export is a separate call in `pages/internal-user/reports/reportExports.js` to `POST /api/internal-user/reports/export/pdf`.

Page	Reads
`ConfigurableReportPage.jsx`	`fetchReport`, `fetchReportFilterOptions`
`ComparisonReportPage.jsx`	`fetchReport`, `fetchComparisonSheets`
`DualPlReportPage.jsx`	`fetchReport`

Page	Reads
`PlAtAGlancePage.jsx`	`fetchReport` , `fetchReportFilterOptions`
`CargoTonnageReportPage.jsx`	`fetchReport`
`VesselYearlyPerformancePage.jsx`	`fetchReport`
`EditableTrackerPage.jsx`	`fetchReport` , then `updateReportTrackerField` on edit
`DailyPositionReportPage.jsx`	Mock file. No API
`AgingReportPage.jsx`	Hard-coded rows in the page. No API
Anything still on `ReportPlaceholder`	Static “not built yet” copy

Masters

Each master is one page, one service, one API prefix under `/api/internal-user/masters/...` .
`MasterModulePage.jsx` only picks the page from the URL. `useMastersModule` only reads the path. It does not load rows.

The page always calls `fetch*` for the list (and `fetch*Lookups` when the form needs dropdowns from other masters). Create, update, status, and delete stay in that same service file.

Page folder	Service	API suffix
`masters/accounting-group`	`accountingGroups.js`	`/accounting-groups`
`masters/accounting-ledger`	`accountingLedgers.js`	`/accounting-ledgers`
`masters/agency-fee`	`agencyFeeRecords.js` (port search: `searchMasterPorts`)	`/api/internal-user/masters` (agency-fee paths live on that base)
`masters/baltic-dry-index`	`balticRoutes.js`	`/baltic-routes`
`masters/bunker-grade`	`bunkerGrades.js`	`/bunker-grades`
`masters/charterer-cost`	`chartererCosts.js`	`/charterer-costs`
`masters/coa-route`	`coaRoutes.js`	`/coa-routes`
`masters/contract-type`	`contractTypes.js`	`/contract-types`
`masters/elibrary-category`	`elibraryCategories.js`	`/elibrary-categories`
`masters/elibrary-reference-type`	`elibraryReferenceTypes.js`	`/elibrary-reference-types`
`masters/expense-type`	`expenseTypes.js`	`/expense-types`
`masters/invoice-status`	`invoiceStatuses.js`	`/invoice-statuses`
`masters/law-arbitration`	`lawArbitrations.js`	`/law-arbitrations`
`masters/load-options`	`loadOptions.js`	`/load-options`
`masters/material`	`materials.js`	`/materials`
`masters/msds`	`msds.js`	`/msds`
`masters/necessary-approval`	`necessaryApprovals.js`	`/necessary-approvals`
`masters/other-miscellaneous-cost`	`otherMiscCosts.js`	`/other-misc-costs`
`masters/other-shipping-cost`	`otherShippingCosts.js`	`/other-shipping-costs`
`masters/owner-related-cost`	`ownerRelatedCosts.js`	`/owner-related-costs`
`masters/panama-canal-capacity-tariff`	`pcctf.js`	`/pcctf`
`masters/panama-canal-fixed-transit`	`pcftf.js`	`/pcftf`
`masters/pc-ums-dwt-ratio`	`estimatedRatios.js`	`/estimated-ratios`

Page folder	Service	API suffix
`masters/port-cost-type`	`portCostTypes.js`	`/port-cost-types`
`masters/port-data`	`portData.js`	`/port-data`
`masters/port-information`	`portInformation.js`	`/port-information`
`masters/rate-net-ton`	`rateNetTons.js`	`/rate-net-tons`
`masters/scnt`	`scnt.js`	`/scnt`
`masters/sdr-rates`	`sdrRates.js`	`/sdr-rates`
`masters/tc-deductions`	`tcDeductions.js`	`/tc-deductions`
`masters/terminal`	`terminals.js`	`/terminals`
`masters/vc-deductions`	`vcDeductions.js`	`/vc-deductions`
`masters/vendor`	`vendors.js`	`/vendors`
`masters/vessel-category`	`vesselCategories.js`	`/vessel-categories`
`masters/vessel-type`	`vesselTypes.js`	`/vessel-types`

Full URL example: vessel types are `GET /api/internal-user/masters/vessel-types`.

E-library, todo, finances, live map, agent portal

Area	Page	Service	API
E-library list	`elibrary/ElibraryPage.jsx`	`elibrary.js` → `fetchElibraryReferences`, `fetchElibraryLookups`	`/api/internal-user/elibrary`
E-library form	`elibrary/ElibraryFormPage.jsx`	`fetchElibraryReference`	same
To-do list	`todo-list/ToDoListPage.jsx`	`todoList.js` → `fetchTodoList`	`/api/internal-user/todo-list`
Voyage search modal	`todo-list/SearchForVoyageModal.jsx`	`searchTodoVoyageByNumber`, `searchTodoVoyagesByVessel`, plus `fetchVcBusinessTypes`	same, and `/api/internal-user/vc` for business types
Combined SOA payable	`combined-soa-payable/CombinedSoaPayablePage.jsx`	`combinedSoaPayable.js`	`/api/internal-user/combined-soa-payable`
Combined SOA payable (TC)	`CombinedSoaPayableTcPage.jsx`	`fetchCombinedSoaPayableTcList`	same
Add group payment	`AddGroupPaymentPage.jsx`	`fetchGroupPaymentLookups`, `fetchGroupPaymentCostLines`, plus generic-finance lookups	combined SOA + `/api/internal-user/generic-finances`
Generic finances list	`generic-finances/GenericFinancesPage.jsx`	`genericFinances.js`	`/api/internal-user/generic-finances`
Add / edit generic invoice	`AddGenericInvoicePage.jsx`	`fetchGenericInvoice`, `fetchGenericInvoiceLookups`, `fetchBankingDetail`, `fetchVendorBanking`	same
Live vessel map	`live-vessel-map/LiveVesselMapPage.jsx`	`live-vessel-map/liveVesselMapApi.js` (not the shared `services/` folder)	Positions: `/api/internal-user/sopf/vessel_positions/within_range`. Route, last position, fleet overlay: `/api/internal-user/live-vessels`
Agent dashboard	`pages/agent/AgentDashboardPage.jsx`	`agentPortal.js`	`GET /api/agent/dashboard`

Area	Page	Service	API
Agent port cost	`AgentPortCostPage.jsx`	`fetchAgentPortCost`	`GET /api/agent/port-cost`
Agent SOF	`AgentSofPage.jsx`	`fetchAgentSof`	`GET /api/agent/sof`

What “mock” means

`backend/src/index.js` logs `using mock data` when `backend/.env` has no database. Many `*Service.js` files branch on `isDbConfigured()` and return fixture rows with `source: 'mock'`. The React page still calls the same service function. You cannot tell mock from database by looking at the component. Check the backend log, or `GET /api/health` (`database.configured`).

Demo charts, the Daily Position report, and the Aging report are different. Those numbers never leave the frontend, even when the database is connected.